

Newer Optimizers:
SOAP MUON MuonClip

Before SOAP: Fast Review of Preconditioner Methods:

- Newton's Method & Hessian Matrix
- Adagrad
- Adam
- Adafactor
- Shampoo
- SOAP

Newton's Method & Hessian Matrix

Let

$$W_t \in \mathbb{R}^{m \times n}, \quad w_t = \text{vec}(W_t) \in \mathbb{R}^d, \quad d = mn.$$

Define the loss function and its derivatives:

$$f(w_t) : \mathbb{R}^d \rightarrow \mathbb{R}, \quad g_t = \nabla f(w_t) \in \mathbb{R}^d, \quad H_t = \nabla^2 f(w_t) \in \mathbb{R}^{d \times d}.$$

Perform a first-order Taylor expansion of the gradient around w_t :

$$\nabla f(w_{t+1}) \approx \nabla f(w_t) + H_t(w_{t+1} - w_t).$$

Setting $\nabla f(w_{t+1}) = 0$:

$$0 = g_t + H_t(w_{t+1} - w_t).$$

Newton update rule:

$$w_{t+1} = w_t - H_t^{-1} g_t.$$

Computational complexity:

$$H_t \in \mathbb{R}^{d \times d}, \quad \text{inversion cost } \mathcal{O}(d^3).$$

Adagrad (Duchi et al., 2011b)

$$G_t = \begin{bmatrix} | & | & & | \\ g_{t,1} & g_{t,2} & \cdots & g_{t,n} \\ | & | & & | \end{bmatrix},$$

$$g_t = \text{vec}(G_t) = \begin{bmatrix} g_{t,1} \\ g_{t,2} \\ \vdots \\ g_{t,n} \end{bmatrix} \in \mathbb{R}^{mn}.$$

$$H_t = H_{t-1} + g_t g_t^\top, \quad H_t \in \mathbb{R}^{mn \times mn}.$$

$$w_{t+1} = w_t - \eta H_t^{-\frac{1}{2}} g_t.$$

$$H_t = \begin{bmatrix} \sum_{\tau=1}^t g_{\tau,1} g_{\tau,1}^\top & \sum_{\tau=1}^t g_{\tau,1} g_{\tau,2}^\top & \cdots & \sum_{\tau=1}^t g_{\tau,1} g_{\tau,n}^\top \\ \sum_{\tau=1}^t g_{\tau,2} g_{\tau,1}^\top & \sum_{\tau=1}^t g_{\tau,2} g_{\tau,2}^\top & \cdots & \sum_{\tau=1}^t g_{\tau,2} g_{\tau,n}^\top \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{\tau=1}^t g_{\tau,n} g_{\tau,1}^\top & \sum_{\tau=1}^t g_{\tau,n} g_{\tau,2}^\top & \cdots & \sum_{\tau=1}^t g_{\tau,n} g_{\tau,n}^\top \end{bmatrix}.$$

Fisher approximation:

$$H \approx \mathbb{E}[g g^\top] \approx F$$

$$F \approx \frac{1}{T} \sum_{t=1}^T g_t g_t^\top$$

⇒ T (const.) is absorbed by learning rate, but why $H_t^{-1/2}$?

Operation	Complexity	Description
Storage of H_t	$O(d^2)$	Full matrix preconditioner
$H_t = H_{t-1} + g_t g_t^\top$	$O(d^2)$	Outer product update
$H_t^{-1/2}$ (matrix inverse sqrt)	$O(d^3)$	Eigen or SVD computation
$H_t^{-1/2} g_t$	$O(d^2)$	Matrix-vector multiplication
Overall per step	$O(d^3)$	Computation
	$O(d^2)$	Storage

Adam (Kingma & Ba, 2015)

$$\text{Adam: } \begin{cases} m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \\ w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \end{cases}$$

If $\beta_1 = 0$, $\beta_2 \rightarrow 1 \Rightarrow$ Adagrad.

Method	Computation	Storage	Notes
Full-Matrix Adagrad	$O(d^3)$	$O(d^2)$	True 2 nd -order preconditioner
Adam	$O(d)$	$O(2d)$	1 st -order, diagonal approx.

Full-Matrix: accurate curvature, high cost. Adam: efficient, smoothed diagonal preconditioner.

Adafactor *(Zhai et al., 2022; Zhao et al., 2024c)*

$$G_t = \begin{bmatrix} \textcolor{red}{g}_{11} & \textcolor{blue}{g}_{12} & \cdots & \textcolor{blue}{g}_{1n} \\ \textcolor{red}{g}_{21} & g_{22} & \cdots & g_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \textcolor{red}{g}_{m1} & g_{m2} & \cdots & g_{mn} \end{bmatrix}, \quad G_t^2 = G_t \odot G_t$$

$$r_t = \frac{1}{n}(G_t \odot G_t)\textcolor{red}{\mathbf{1}}_n, \quad \textcolor{blue}{c}_t^\top = \frac{1}{m}\textcolor{blue}{\mathbf{1}}_m^\top (G_t \odot G_t)$$

$$\textcolor{red}{\mathbf{1}}_n = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}_{n \times 1}, \quad \textcolor{blue}{\mathbf{1}}_m^\top = [1 \quad 1 \quad \cdots \quad 1]_{1 \times m}$$

$$\hat{V}_t = \frac{r_t c_t^\top}{\text{mean}(r_t)}$$

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) G_t$$

$$\hat{G}_t = \frac{M_t}{\sqrt{\hat{V}_t} + \epsilon}$$

$$W_{t+1} = W_t - \eta \hat{G}_t$$

$$V_t \approx \textcolor{red}{r}_t \textcolor{blue}{c}_t^\top, \quad H_t \approx \text{diag}(\textcolor{red}{r}_t)^{1/2} \text{diag}(\textcolor{blue}{c}_t)^{1/2}, \quad \text{Storage: } O(m + n)$$

Shampoo (Gupta et al., 2018b)

matrix case:

$$L_t = \sum_{\tau=1}^t G_\tau G_\tau^\top$$

$$R_t = \sum_{\tau=1}^t G_\tau^\top G_\tau$$

$$(L_t \otimes R_t^\top) \text{vec}(G_t) = \text{vec}(L_t G_t R_t)$$

$$W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}$$

$$\text{vec}(W_{t+1}) = \text{vec}(W_t) - \eta H_t^{-1} \text{vec}(G_t)$$

$$H_t = L_t^{1/4} \otimes R_t^{1/4}$$

$$H_t^{-1} = (L_t^{1/4} \otimes R_t^{1/4})^{-1} \approx (\mathbb{E}[G_t G_t^\top])^{-1/2}$$

(Note: H_t^{-1} here corresponds to the $H^{-1/2}$ term in Adagrad.)

Operation (per step)	Compute Complexity	Memory Complexity
Compute gradient $G_t \in \mathbb{R}^{m \times n}$	$O(mn)$	$O(mn)$
Update $L_t = \sum_{\tau=1}^t G_\tau G_\tau^\top$	$O(mn^2)$	$O(m^2)$
Update $R_t = \sum_{\tau=1}^t G_\tau^\top G_\tau$	$O(m^2 n)$	$O(n^2)$
Matrix inverse roots $L_t^{-1/4}, R_t^{-1/4}$	$O(m^3 + n^3)$	$O(m^2 + n^2)$
Preconditioned update $W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}$	$O(mn(m+n))$	$O(mn)$
Total (per step)	$O(m^3 + n^3 + mn(m+n))$	$O(m^2 + n^2 + mn)$

Fisher Approximation: $H_t = \mathbb{E}_{x,y}[\nabla_W^2 \ell(W_t; x, y)] \approx \mathbb{E}_{x,y}[\nabla_W \ell(W_t; x, y) \nabla_W \ell(W_t; x, y)^\top] = F(W_t).$

$$F(W_t) = \mathbb{E}_{x \sim \mathcal{D}, y \sim p(y|x; W_t)} [\nabla_W \log p(y|x; W_t) \nabla_W \log p(y|x; W_t)^\top].$$

Approximation holds when $\ell(W; x, y) = -\log p(y|x; W)$, $y \sim p(y|x; W_t)$, and the model is locally well-specified.

At the optimum W^* , $H_t = F(W^*)$.

New era: Why **Large-Batch** Optimizers Matter

Parameter Scaling of Major Language Models (2018–2025)			
Year	Model	Parameters	Scale (approx.)
2018	GPT-1	0.12 B	1.2×10^8
2019	GPT-2	1.5 B	1.5×10^9
2020	GPT-3	175 B	1.75×10^{11}
2022	PaLM (Google)	540 B	5.4×10^{11}
2024	LLaMA-3.1 (Meta)	405 B	4.05×10^{11}
2025	LLaMA-4 Maverick (Meta)	400 B	4.0×10^{11}
2025	Qwen3-Max (Alibaba)	1.0 T	1.0×10^{12}
2025	Kimi K2 (Moonshot)	1.0 T	1.0×10^{12}

- The **parameter scale** of large language models has increased by several orders of magnitude, raising higher demands on the **efficiency** and **stability** of optimization algorithms.
- Traditional optimizers are **constrained** by storage and computational complexity, often relying on step-by-step or small-batch updates, which lead to low training efficiency.
- Under the **necessity** of **large-batch learning**, new optimizers propose methods to **reduce memory** and **computational costs**, making large-batch and curvature-aware optimization widely applicable.

ShampoO with Adam in Preconditioner's eigenbasis (SOAP)

Outline:

- The Muon Optimizer (*K. Jordan et al. 2024*)
- SVD vs NS
- Why MUON can't scale up without AadmW

Approach: ShampoO with Adam in Preconditioner's eigenbasis (SOAP)

Algorithm 1 Single step of idealized Shampoo with power 1/2.

- 1: Sample batch B_t .
 - 2: $G_t \in \mathbb{R}^{m \times n} \leftarrow -\nabla_W \phi_{B_t}(W_t)$
 - 3: $L \leftarrow \mathbb{E}_B[G_B G_B^\top]$ {Where the expectation is over a random batch B .}
 - 4: $R \leftarrow \mathbb{E}_B[G_B^\top G_B]$
 - 5: $\hat{H} \leftarrow L \otimes R / \text{Trace}(L)$
 - 6: $W_t \leftarrow W_{t-1} - \eta \hat{H}^{-1/2} G_t = W_{t-1} - \eta L^{-1/2} G_t R^{-1/2} / \text{Trace}(L)^{-1/2}$
-

1. Step Accumulation vs Batch Expectation

2. Why is $H_B^{-1} \approx L_B^{-1/4} \otimes R_B^{-1/4}$ still valid under expectation?

$$H_B^{-1} = \mathbb{E}_B \left[(G_B G_B^\top)^{-1/4} \otimes (G_B^\top G_B)^{-1/4} \right] \approx \mathbb{E}_B[G_B G_B^\top]^{-1/4} \otimes \mathbb{E}_B[G_B^\top G_B]^{-1/4}.$$

Use Shampoo to find differences:

$$L_t = \sum_{\tau=1}^t G_\tau G_\tau^\top,$$

$$R_t = \sum_{\tau=1}^t G_\tau^\top G_\tau,$$

$$H_t = L_t^{1/4} \otimes R_t^{1/4},$$

$$H_t^{-1} = L_t^{-1/4} \otimes R_t^{-1/4}.$$

$$\text{vec}(W_{t+1}) = \text{vec}(W_t) - \eta H_t^{-1} \text{vec}(G_t) = \text{vec}(W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}).$$

This approximation assumes that row- and column-wise gradient covariances are weakly correlated,

i.e., $\mathbb{E}[A \otimes B] \approx \mathbb{E}[A] \otimes \mathbb{E}[B]$, when A, B are statistically separable across dimensions.

$$H_B^{-1} \text{vec}(G_B) \approx (L_B^{-1/4} \otimes R_B^{-1/4}) \text{vec}(G_B) = \text{vec}(L_B^{-1/4} G_B R_B^{-1/4}).$$

3. Connection to Adagrad.

$$H_{\text{Shampoo}}^{-1} = (L^{-1/4} \otimes R^{-1/4}) \approx H_{\text{Adagrad}}^{-1/2}.$$

If H_{Adagrad} uses exponent (-1) , then it corresponds to $(H_{\text{Shampoo}}^2)^{-1}$.

Hence, in SOAP the preconditioner adopts $L^{-1/2}$, $R^{-1/2}$ to match the Adagrad curvature scaling.

4. From Adagrad to SOAP: Smoothing and Expectation Stability.

In Adagrad (online learning), each sample contributes a noisy gradient,

so the accumulated preconditioner $H = \sum_t g_t g_t^\top$ is highly stochastic.

To stabilize updates, Adagrad applies a square-root scaling: $H^{-1/2}$, which smooths the variance of g_t .

In SOAP, gradients are estimated over batches: $G_B = \nabla_W \ell(W; B)$, $H_B = \mathbb{E}_B[G_B G_B^\top]$.

Under expectation, the noise term averages out ($\mathbb{E}[\xi] = 0$), yielding a more stable curvature estimate.

Thus, no extra square-root smoothing is required: H_B^{-1} (via $L_B^{-1/2}$, $R_B^{-1/2}$) is sufficient,

leading to both lower computation cost and smoother optimization dynamics.

Approach: ShampoO with Adam in Preconditioner's eigenbasis (**SOAP**)

Algorithm 1 Single step of idealized Shampoo with power $1/2$.

- 1: Sample batch B_t .
 - 2: $G_t \in \mathbb{R}^{m \times n} \leftarrow -\nabla_W \phi_{B_t}(W_t)$
 - 3: $L \leftarrow \mathbb{E}_B[G_B G_B^T]$ {Where the expectation is over a random batch B .}
 - 4: $R \leftarrow \mathbb{E}_B[G_B^T G_B]$
 - 5: $\hat{H} \leftarrow L \otimes R / \text{Trace}(L)$
 - 6: $W_t \leftarrow W_{t-1} - \eta \hat{H}^{-1/2} G_t = W_{t-1} - \eta L^{-1/2} G_t R^{-1/2} / \text{Trace}(L)^{-1/2}$
-

Algorithm 2 Single step of idealized Adafactor in Shampoo's eigenspace.

- 1: Sample batch B_t .
 - 2: $G_t \in \mathbb{R}^{m \times n} \leftarrow -\nabla_W \phi_{B_t}(W_t)$
 - 3: $L \leftarrow \mathbb{E}_B[G_B G_B^T]$
 - 4: $R \leftarrow \mathbb{E}_B[G_B^T G_B]$
 - 5: $Q_L \leftarrow \text{Eigenvectors}(L)$
 - 6: $Q_R \leftarrow \text{Eigenvectors}(R)$
 - 7: $G'_t \leftarrow Q_L^T G_t Q_R$
 - 8: {Idealized version of code for Adafactor taking G'_t to be the gradient}
 - 9: $G'_B \leftarrow Q_L^T G_B Q_R$
 - 10: $A = \mathbb{E}_B[G'_B \odot G'_B] \mathbf{1}_m$ where $G'_B = Q_L^T G_B Q_R$
 - 11: $C = \mathbf{1}_n^T \mathbb{E}_B[G'_B \odot G'_B]$
 - 12: $\hat{V}_t = \frac{A C^T}{\mathbf{1}_n^T A}$ {Elementwise division}
 - 13: $G''_t \leftarrow \frac{G'_t}{\sqrt{\hat{V}_t + \epsilon}}$ {Elementwise division and square root}
 - 14: $G'''_t \leftarrow Q_L G''_t Q_R^T$ {Projecting back to original space}
 - 15: $W_t \leftarrow W_{t-1} - \eta G'''_t$
-

Low-rank variance approximation in Adafactor form:

$$A = \mathbb{E}_B[(G'_B \odot G'_B) \mathbf{1}_n], \quad C^\top = \mathbb{E}_B[\mathbf{1}_m^\top (G'_B \odot G'_B)],$$

$$G'_B = \begin{bmatrix} g'_{11} & g'_{12} & \cdots & g'_{1n} \\ g'_{21} & g'_{22} & \cdots & g'_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ g'_{m1} & g'_{m2} & \cdots & g'_{mn} \end{bmatrix}, \quad \mathbf{1}_n = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}_{n \times 1}, \quad \mathbf{1}_m^\top = [1 \quad 1 \quad \cdots \quad 1]_{1 \times m}.$$

$$A = \begin{bmatrix} \sum_{j=1}^n g'_{1j}{}^2 \\ \sum_{j=1}^n g'_{2j}{}^2 \\ \vdots \\ \sum_{j=1}^n g'_{mj}{}^2 \end{bmatrix}_{m \times 1}, \quad C^\top = [\sum_{i=1}^m g'_{i1}{}^2 \quad \sum_{i=1}^m g'_{i2}{}^2 \quad \cdots \quad \sum_{i=1}^m g'_{in}{}^2]_{1 \times n}.$$

$$\hat{V}_t = \frac{A C^\top}{\mathbf{1}_m^\top A} \quad (\hat{V}_t \in \mathbb{R}^{m \times n})$$

Algorithm 3 Single step of SOAP for a $m \times n$ layer. Per layer, we maintain four matrices: $L \in \mathbb{R}^{m \times m}$, $R \in \mathbb{R}^{n \times n}$ and $V, M \in \mathbb{R}^{m \times n}$. For simplicity we ignore the initialization and other boundary effects such as bias correction. Hyperparameters: Learning rate η , betas = (β_1, β_2) , epsilon ϵ , and preconditioning frequency f .

An implementation of SOAP is available at <https://github.com/nikhilvyas/SOAP/tree/main>.

```
1: Sample batch  $B_t$ .
2:  $G \in \mathbb{R}^{m \times n} \leftarrow -\nabla_W \phi_{B_t}(W_t)$ 
3:  $G' \leftarrow Q_L^T G Q_R$ 
4:  $M \leftarrow \beta_1 M + (1 - \beta_1) G$ 
5:  $M' \leftarrow Q_L^T M Q_R$ 
6: {Now we “run” Adam on  $G'$ }
7:  $V \leftarrow \beta_2 V + (1 - \beta_2)(G' \odot G')$  {Elementwise multiplication}
8:  $N' \leftarrow \frac{M'}{\sqrt{\hat{V}_t + \epsilon}}$  {Elementwise division and square root}
9: {Now that we have preconditioned by Adam in the rotated space, we go back to the original space.}
10:  $N \leftarrow Q_L N' Q_R^T$ 
11:  $W \leftarrow W - \eta N$ 
12: {End of gradient step, we now update  $L$  and  $R$  and possibly also  $Q_L$  and  $Q_R$ . }
13:  $L \leftarrow \beta_2 L + (1 - \beta_2) G G^T$ 
14:  $R \leftarrow \beta_2 R + (1 - \beta_2) G^T G$ 
15: if  $t \% f == 0$  then
16:    $Q_L \leftarrow \text{Eigenvectors}(L, Q_L)$ 
17:    $Q_R \leftarrow \text{Eigenvectors}(R, Q_R)$ 
18: end if
```

Algorithm 4 Eigenvectors function, implemented using power iteration and QR decomposition. Inputs: PSD matrix P and estimate of eigenvectors Q . If the estimate was exact we would have $P = Q D Q^T$ where D is the diagonal matrix with eigenvalues.

```
1:  $S \leftarrow P Q$ 
2:  $Q \leftarrow \text{QR}(S)$ 
```

Why use shampoo's eigenspace?
Project it there and project it
back, any benefits?

Performance: Shampoo with Adam in Preconditioner's eigenbasis (**SOAP**)

7.2 SPACE USAGE OF SOAP

For a $m \times n$ matrix where $m > n$ we require

$$2m^2 \text{ (for } L, Q_L) + 2n^2 \text{ (for } R, Q_R) + 3mn \text{ (for gradient, } M, V)$$

space usage⁴ (beyond weights and activations), specifically for L, Q_L, R, Q_R , momentum (M), AdamW's second order estimate (V), and the gradient. This is the same space usage as DistributedShampoo while AdamW uses $3mn$.

7.3 TIME OVERHEAD OF SOAP

There are two types of overhead of Shampoo and SOAP over AdamW: the overhead per step and the overhead when changing the preconditioner (or for SOAP, the preconditioner's eigenbasis). Let us first analyze the first one. For SOAP per step for a layer of size $m \times n$ we have an overhead of

$$m^3 \text{ (updating } L) + n^3 \text{ (updating } R) + (2m^2n + 2mn^2) \text{ (projecting and projecting back on both sides).}$$

We note that this is more than the overhead of Shampoo which is $m^3 + n^3 + m^2n + n^2m$. This can be observed in Figure 2 (bottom, right) but not in the other figures since there the second type of overhead is the dominant term.

Performance of SOAP: some notable experiment results

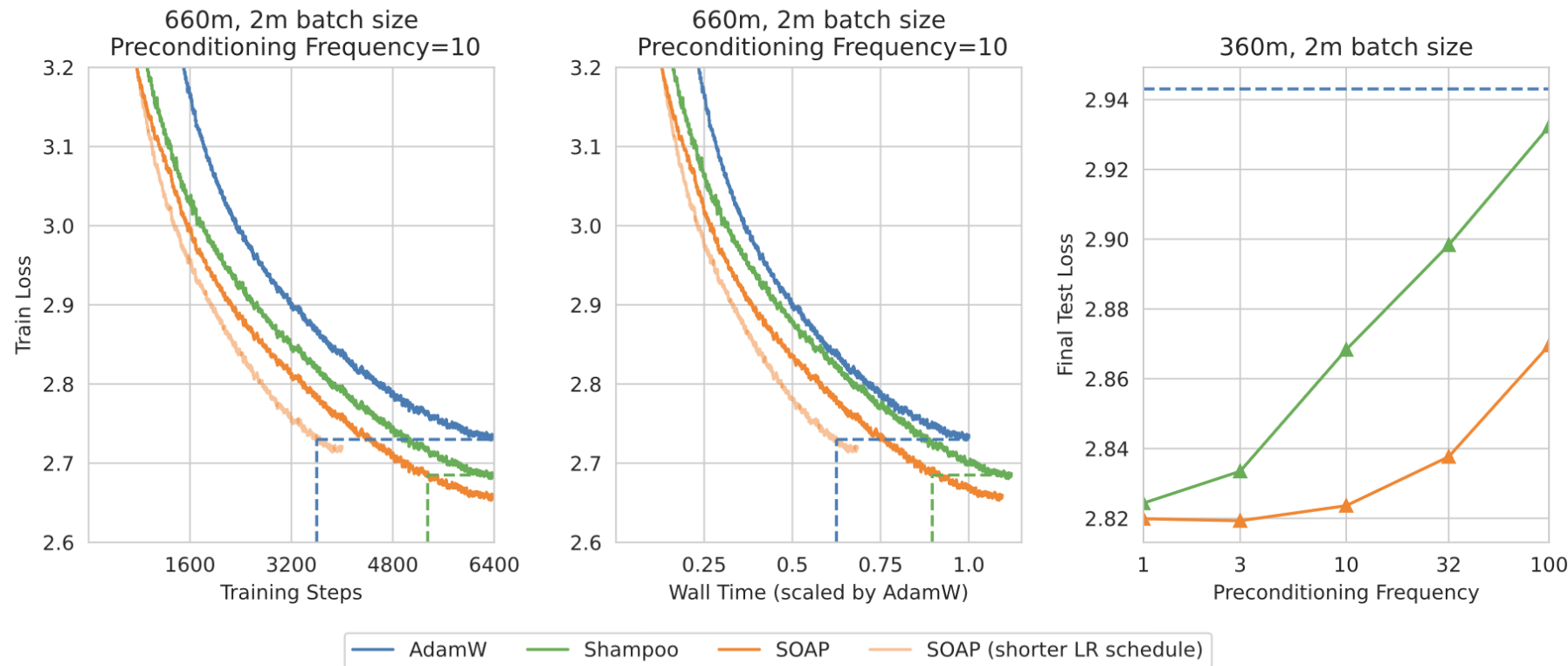


Figure 1: Comparing performance of tuned runs for AdamW, Shampoo (using DistributedShampoo (Shi et al., 2023) implementation) and SOAP. In left and middle figures, Shampoo and SOAP use a preconditioning frequency of 10. The "shorter LR schedule" plot is where we tuned the cosine decay so as to achieve the same terminal performance as AdamW. There we observe a $\geq 40\%$ reduction in the number of iterations and a $\geq 35\%$ reduction in wall clock time compared to AdamW, and approximately a 20% reduction in both metrics compared to Shampoo. In the right figure we ablate preconditioning frequency and observe a slower degradation of performance of SOAP as compared to Shampoo. See Section 6 for a discussion of experimental results and ablation of batch size and Section 5 for experimental methodology.

Performance of SOAP: some notable experiment results

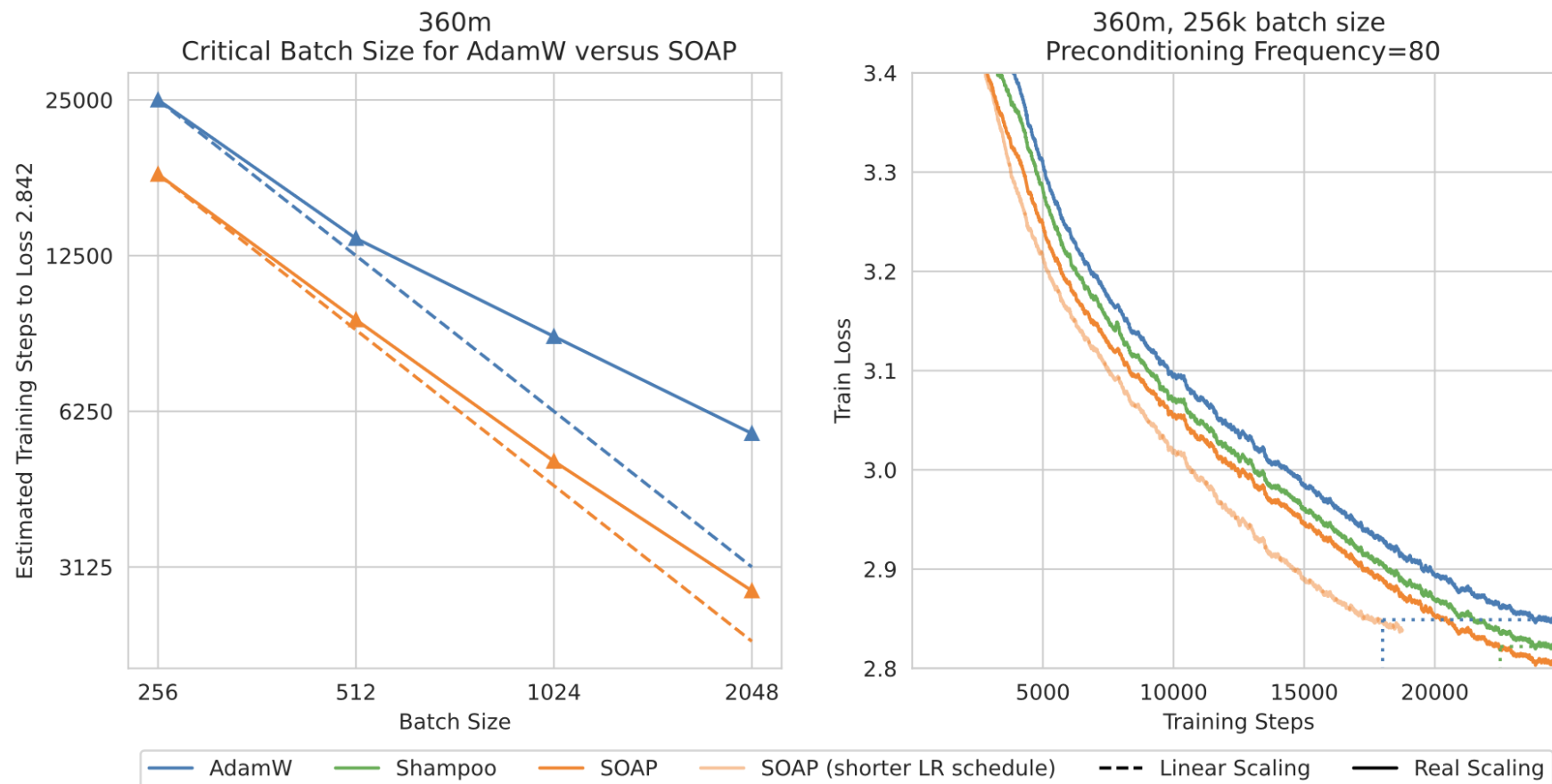


Figure 4: (left) Comparing the critical batch size of AdamW vs SOAP. We can see that SOAP improves the critical batch size, by being much closer to the ideal linear scaling with batch size as compared to AdamW. (right) Comparing performance of tuned runs for AdamW, Shampoo (using DistributedShampoo (Shi et al., 2023) implementation) and SOAP for token batch size of 256k. Shampoo and SOAP use preconditioning frequency of 80. We observe a $\geq 25\%$ reduction in the number of iterations compared to AdamW, and approximately a 10% reduction compared to Shampoo. See Figure 2 (right) for wall-clock time improvement and Section 5 for detailed calculation of efficiency improvement.

MUON is Scalable for LLM Training

Outline:

- The Muon Optimizer (*K. Jordan et al. 2024*)
- SVD vs NS
- Why MUON perform bad when scale up without Weight Decay

- **The Muon Optimizer Muon** (*K. Jordan et al. 2024*)

Muon update rule:
$$\begin{cases} \mathbf{M}_t = \mu \mathbf{M}_{t-1} + \nabla \mathcal{L}_t(\mathbf{W}_{t-1}), \\ \mathbf{O}_t = (\mathbf{M}_t \mathbf{M}_t^\top)^{-1/2} \mathbf{M}_t, \\ \mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t \mathbf{O}_t. \end{cases}$$

Goal: obtain $\mathbf{O}_t =$ polar factor of $\mathbf{M}_t$.

Understanding from the SVD Perspective

$$\mathbf{M}_t = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top, \quad \mathbf{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r),$$

$$\Rightarrow (\mathbf{M}_t \mathbf{M}_t^\top)^{-1/2} \mathbf{M}_t = (\mathbf{U} \mathbf{\Sigma}^2 \mathbf{U}^\top)^{-1/2} \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top = \mathbf{U} \mathbf{V}^\top.$$

Why not SVD?

Computational Complexity:

Full SVD requires $O(\min(mn^2, m^2n))$ operations.

Interpretation:

- The momentum matrix $\mathbf{M}_t$ is decomposed into rotation ($\mathbf{U} \mathbf{V}^\top$) and scaling ($\mathbf{\Sigma}$) components.
- Removing $\mathbf{\Sigma}$ cancels singular value magnitudes, keeping only the orthogonal transformation $\mathbf{U} \mathbf{V}^\top$.

- **The Muon Optimizer Muon** (*K. Jordan et al. 2024*)

Newton–Schulz Iteration for Orthogonalization

Goal: $\mathbf{O}_t = (\mathbf{M}_t \mathbf{M}_t^\top)^{-1/2} \mathbf{M}_t.$

Initialize: $\mathbf{X}_0 = \frac{\mathbf{M}_t}{\|\mathbf{M}_t\|_F}.$

Iterate: $\mathbf{X}_{k+1} = a \mathbf{X}_k + b (\mathbf{X}_k \mathbf{X}_k^\top) \mathbf{X}_k + c (\mathbf{X}_k \mathbf{X}_k^\top)^2 \mathbf{X}_k, \quad k = 0, 1, \dots, N.$

After N steps: $\mathbf{O}_t \approx \mathbf{X}_N.$

Idea:

- Newton–Schulz approximates $(\mathbf{M} \mathbf{M}^\top)^{-1/2}$ by repeated matrix multiplications.
- Coefficients a, b, c are chosen so that the polynomial $f(z) = az + bz^3 + cz^5$ has a fixed point at $z = 1$ and contracts near it.
- Normalization by $\|\mathbf{M}_t\|_F$ ensures all eigenvalues λ_i of $\mathbf{M}_t \mathbf{M}_t^\top$ are close to 1, guaranteeing convergence.

Goods:

- No SVD
- GPU-friendly (GEMM)

Cons:

- Bf16 RMS problem
- Parameter sensitive: a, b, c

- Scaling up MUON

Weight Decay While Muon performs significantly better than AdamW on a small scale as shown by K. Jordan et al. 2024, we found the performance gains diminish when we scale up to train a larger model with more tokens. We observed that both the weight and the layer output's RMS keep growing to a large scale, exceeding the high-precision range of bf16, which might hurt the model's performance. To resolve this issue, we introduced the standard AdamW (Loshchilov et al. 2019) weight decay mechanism into Muon².

$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(\mathbf{O}_t + \lambda \mathbf{W}_{t-1}) \quad (3)$$

Lets try without /with weight decay

Muon Update without Weight Decay: $W_t = W_{t-1} - \eta_t O_t$, $O_t \approx (M_t M_t^\top)^{-1/2} M_t$.

Energy Evolution: $E_t = \|W_t\|_F^2 = \|W_{t-1} - \eta_t O_t\|_F^2 = \|W_{t-1}\|_F^2 - 2\eta_t \langle W_{t-1}, O_t \rangle + \eta_t^2 \|O_t\|_F^2$.

If O_t and W_{t-1} are approximately uncorrelated ($\mathbb{E}[\langle W_{t-1}, O_t \rangle] \approx 0$), then

$$\mathbb{E}[E_t] \approx E_{t-1} + \eta_t^2 \|O_t\|_F^2 \Rightarrow E_t \propto t \Rightarrow \|W_t\|_F \propto \sqrt{t}.$$

Without weight decay, $\text{RMS}(W_t)$ grows unboundedly.

$$\begin{aligned} \text{RMS}(G) &= \sqrt{\frac{1}{AB} \sum_{i=1}^A \sum_{j=1}^B g_{ij}^2} \\ &= \sqrt{\frac{1}{AB} \text{tr}(GG^\top)} \\ &= \frac{\|G\|_F}{\sqrt{AB}}, \end{aligned}$$

where $\|G\|_F = \sqrt{\text{tr}(GG^\top)}$ is the Frobenius norm of G .

Muon Update with AdamW-style Weight Decay: $W_t = W_{t-1} - \eta_t(O_t + \lambda W_{t-1})$.

$$E_t = \|(1 - \eta_t \lambda)W_{t-1} - \eta_t O_t\|_F^2 = (1 - \eta_t \lambda)^2 E_{t-1} - 2\eta_t(1 - \eta_t \lambda) \langle W_{t-1}, O_t \rangle + \eta_t^2 \|O_t\|_F^2.$$

Taking expectation and ignoring cross-terms:

$$\mathbb{E}[E_t] \approx (1 - \eta_t \lambda)^2 E_{t-1} + \eta_t^2 \|O_t\|_F^2.$$

At steady state ($E_t \approx E_{t-1} = E^*$),

$$E^*[1 - (1 - \eta_t \lambda)^2] \approx \eta_t^2 \|O_t\|_F^2 \Rightarrow 2\eta_t \lambda E^* \approx \eta_t^2 \|O_t\|_F^2.$$

$$E^* \approx \frac{\eta_t}{2\lambda} \|O_t\|_F^2.$$

Since $\|O_t\|_F^2 = r$ (rank of O_t),

$$\text{RMS}(W_t) = \frac{\sqrt{E^*}}{\sqrt{AB}} \approx \frac{1}{\sqrt{2\lambda}} \cdot \frac{\sqrt{\eta_t}}{\sqrt{\max(A, B)}}.$$

With weight decay, $\text{RMS}(W_t)$ stabilizes at a finite equilibrium: $\text{RMS}(W_t) \propto \sqrt{\frac{\eta_t}{\lambda \max(A, B)}}$.

Computation Complexity:

Dominated by Muon: $O(m^2 n)$,

Newton–Schulz iteration (N steps): $O(Nm^3)$,

Weight decay term: $O(mn)$ (negligible).

Time: $O(m^2 n + Nm^3)$ ($\approx O(m^2 n)$ for practical N).

Memory Complexity:

$W_t, M_t, O_t \in \mathbb{R}^{m \times n} \Rightarrow 3mn$,

$M_t M_t^\top \in \mathbb{R}^{m \times m} \Rightarrow m^2$,

(optional cache O_{t-1}).

Storage: $3mn + m^2$.

KIMI K2: open agentic intelligence

- MUON with QK-clip in pre-training: 15.5 trillion tokens with zero loss spike
- Post-training:
large-scale agentic data synthesis pipeline + reinforcement learning

MUON with QK-clip in pre-training

QK-Clip: Rescaling Query and Key Weights to Constrain Logits

$$Q^{(h)} = XW_q^{(h)}, \quad K^{(h)} = XW_k^{(h)}, \quad V^{(h)} = XW_v^{(h)}.$$

$$O^{(h)} = \text{softmax}\left(\frac{1}{\sqrt{d}} Q^{(h)} K^{(h)\top}\right) V^{(h)}.$$

$$S_{\max}^{(h)} = \frac{1}{\sqrt{d}} \max_{X \in B} \max_{i,j} Q_i^{(h)} K_j^{(h)\top}.$$

Rescaling rule: $W_q^{(h)} \leftarrow \gamma^\alpha W_q^{(h)}, \quad W_k^{(h)} \leftarrow \gamma^{1-\alpha} W_k^{(h)},$

$$\gamma = \min\left(1, \frac{\tau}{S_{\max}}\right), \quad S_{\max} = \max_h S_{\max}^{(h)}, \quad \alpha = 0.5.$$

Interpretation: $\begin{cases} S_{\max}^{(h)} > \tau \Rightarrow \text{rescale both } W_q^{(h)}, W_k^{(h)} \text{ to reduce spectral norm} \\ \text{No effect on current forward/backward pass;} \\ \text{Acts as dynamic bound on logit energy growth.} \end{cases}$

Algorithm 1 MuonClip Optimizer

```

1: for each training step  $t$  do
2:   // 1. Muon optimizer step
3:   for each weight  $\mathbf{W} \in \mathbb{R}^{n \times m}$  do
4:      $\mathbf{M}_t = \mu \mathbf{M}_{t-1} + \mathbf{G}_t$   $\triangleright \mathbf{M}_0 = \mathbf{0}, \mu = 0.9$ 
5:      $\mathbf{O}_t = \text{Newton-Schulz}(\mathbf{M}_t) \cdot \sqrt{\max(n, m)} \cdot 0.2$ 
6:      $\mathbf{W}_t = \mathbf{W}_{t-1} - \eta(\mathbf{O}_t + \lambda \mathbf{W}_{t-1})$ 
7:   end for
8:   // 2. QK-Clip
9:   for each attention head  $h$  in every attention layer of the model do
10:    Obtain  $S_{\max}^h$  already computed during forward
11:    if  $S_{\max}^h > \tau$  then
12:       $\gamma \leftarrow \tau / S_{\max}^h$ 
13:       $\mathbf{W}_{qc}^h \leftarrow \mathbf{W}_{qc}^h \cdot \sqrt{\gamma}$ 
14:       $\mathbf{W}_{kc}^h \leftarrow \mathbf{W}_{kc}^h \cdot \sqrt{\gamma}$ 
15:       $\mathbf{W}_{qr}^h \leftarrow \mathbf{W}_{qr}^h \cdot \gamma$ 
16:    end if
17:  end for
18: end for

```

Why Muon is More Prone to Logit Explosion

Logit Explosion: $S_{\max} = \max_{i,j} (q_i \cdot k_j)$ grows unbounded during training.

$$|q_i \cdot k_j| \leq \|q_i\| \|k_j\| \leq \|x_i\| \|x_j\| \|W_q\| \|W_k\|,$$

$\Rightarrow$ Explosion mainly driven by growing spectral norms of W_q, W_k .

SVD formulation: $W_{t-1} = \sum_i \sigma_i u_i v_i^\top$

Muon update: $\Delta W_t = \sum_j \bar{\sigma}_j \bar{u}_j \bar{v}_j^\top, \quad W_t = \sum_i \sigma_i u_i v_i^\top + \sum_j \bar{\sigma}_j \bar{u}_j \bar{v}_j^\top.$

Muon produces updates with nearly equal singular values, Therefore:

In Muon, $u_i v_i^\top$ and $\bar{u}_j \bar{v}_j^\top$ align more easily, $\Rightarrow$ the singular values of W_t grow additively.

Result: $\|W_q W_k^\top\|_2 \uparrow \Rightarrow S_{\max} \uparrow \Rightarrow \text{softmax}(QK^\top)$ becomes unstable (logit explosion).

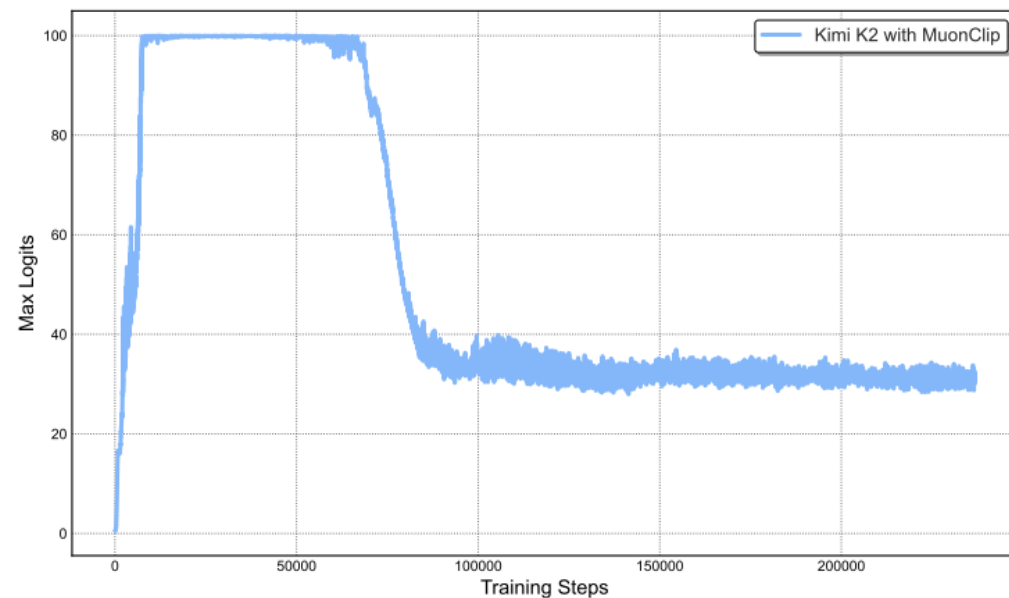
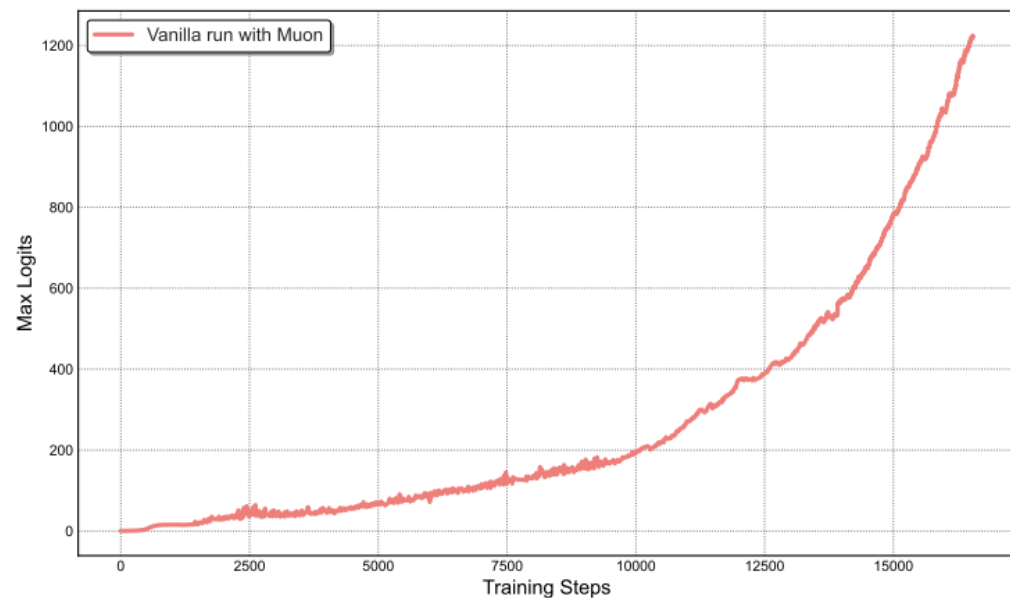


Figure 2: Left: During a mid-scale training run, attention logits rapidly exceed 1000, which could lead to potential numerical instabilities and even training divergence. Right: Maximum logits for Kimi K2 with MuonClip and $\tau = 100$ over the entire training run. The max logits rapidly increase to the capped value of 100, and only decay to a stable range after approximately 30% of the training steps, demonstrating the effective regulation effect of QK-Clip.